

CCS 3105: Systems Programming

Week 3 Practical Session

Linux Processes and Process Management

Duration: 2 Hours

Course: CCS 3105 – Systems Programming

Practical Title

Working with Linux Processes Using `fork()`, `getpid()`, `wait()`, and `exec()` System Calls

Practical Learning Outcomes

By the end of this practical session, students should be able to:

1. Explain the concept of a process in Linux.
 2. Create child processes using `fork()`.
 3. Distinguish between parent and child processes.
 4. Obtain process identifiers (PIDs).
 5. Synchronize parent and child processes using `wait()`.
 6. Replace a process image using the `exec()` family of system calls.
 7. Observe process creation and termination using Linux commands.
 8. Debug common process-related programming errors.
-

Software Requirements

Students should have access to:

- Ubuntu/Linux
 - GCC Compiler
 - Terminal
 - Visual Studio Code / Nano / Vim
 - Process monitoring tools (`ps`, `top`, `pstree`)
-

Part A: Exploring Linux Processes (15 Minutes)

Exercise 1: Viewing Running Processes

Open a terminal and execute:

ps

Observe the output.

Now display detailed process information:

ps -ef

Display processes in a hierarchical tree:

pstree

Monitor running processes dynamically:

top

(or)

htop

if installed.

Discussion Questions

1. What is a process?
2. What information does the operating system maintain for each process?
3. What is the purpose of the Process ID (PID)?
4. Why are process trees useful?

Part B: Obtaining Process Information (15 Minutes)

Exercise 2

Create the following program.

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Current Process ID : %d\n", getpid());
    printf("Parent Process ID : %d\n", getppid());

    return 0;
}
```

Compile

```
gcc processinfo.c -o processinfo
```

Execute

```
./processinfo
```

Expected Output

Current Process ID : 3245

Parent Process ID : 3101

Questions

1. Explain the difference between `getpid()` and `getppid()`.
 2. Why does every process have a unique PID?
-

Part C: Creating Child Processes Using `fork()` (30 Minutes)

Exercise 3

Write the following program.

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;
    pid = fork();
    if(pid == 0)
    {
        printf("Child Process\n");
        printf("PID = %d\n", getpid());
    }
    else
    {
        printf("Parent Process\n");
        printf("PID = %d\n", getpid());
    }

    return 0;
}
```

Compile

```
gcc fork1.c -o fork1
```

Run

./fork1

Expected Output

Parent Process

PID = 3500

Child Process

PID = 3501

(The order may vary.)

Discussion

Explain:

fork()

↓

Parent

Child

Both continue executing independently.

Questions

1. How many processes exist after fork()?
 2. What value does fork() return in the parent?
 3. What value does fork() return in the child?
-

Part D: Understanding Process Duplication (15 Minutes)

Exercise 4

Modify the previous program.

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main()
```

```
{
```

```
int number = 100;

if(fork()==0)
{
    number = 200;
    printf("Child: %d\n",number);
}
else
{
    printf("Parent: %d\n",number);
}
return 0;
}
```

Expected Output

Parent: 100

Child: 200

Discussion

Does the child changing the variable affect the parent?

Explain why.

Part E: Parent Waiting for Child (20 Minutes)

Exercise 5

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    pid = fork();
    if(pid==0)
    {
        printf("Child running...\n");
        sleep(3);
        printf("Child finished.\n");
    }
}
```

```
else
{
    wait(NULL);
    printf("Parent resumes.\n");
}
return 0;
}
```

Expected Output

Child running...

Child finished.

Parent resumes.

Discussion

Why is wait() necessary?

Part F: Executing Another Program (exec()) (20 Minutes)

Exercise 6

```
#include <stdio.h>
#include <unistd.h>
```

```
int main()
{
    printf("Before exec\n");
    execl("/bin/ls", "ls", "-l", NULL);
    printf("After exec\n");
    return 0;
}
```

Compile

Execute

Observe

Only

Before exec followed by directory listing.

Discussion

Why is

After exec

never displayed?

Explain how `exec()` replaces the current process image.

Part G: Creating Multiple Child Processes (20 Minutes)

Exercise 7

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    for(int i=0;i<3;i++)
    {
        fork();
        printf("PID=%d\n",getpid());
    }
    return 0;
}
```

Observe

Many more processes are created.

Questions

1. How many processes were created?
 2. Explain why.
 3. Draw the resulting process tree.
-

Part H: Process Monitoring (15 Minutes)

While Exercise 5 is running,

open another terminal.

Run

ps -ef

Locate

Parent process.

Locate

Child process.

Now execute

ps

Observe

Relationship between processes.

Part I: Error Handling (15 Minutes)

Modify

Exercise 3.

```
pid = fork();
```

```
if(pid<0)
{
    perror("Fork Failed");
    return 1;
}
```

Explain

Why should every call to

fork()

be checked?

Challenge Exercise 1

Develop a program that

Creates

Three child processes.

Each child displays

Child Number

PID

Parent PID

Parent waits until all children finish.

Challenge Exercise 2

Write a program that

Creates a child.

The child executes

date

using

execl()

Parent waits.

Displays

Child Completed Successfully

Challenge Exercise 3

Develop a process tree similar to

Parent

├── Child 1

├── Child 2

└── Child 3

Each child prints

PID

PPID

Termination message.

Reflection Questions

1. Explain the purpose of fork().
2. Why does fork() return different values?

3. Explain the difference between `fork()` and `exec()`.
 4. Why is `wait()` important?
 5. Explain why parent and child processes do not share ordinary variables after `fork()`.
 6. What happens if the parent terminates before the child?
 7. What is a zombie process?
 8. What is an orphan process?
-

Individual Assessment

Write a complete C program that performs the following:

1. Displays the parent's PID.
 2. Creates a child process.
 3. The child displays:
 - Child PID
 - Parent PID
 - A message indicating it is executing.
 4. The parent waits for the child to complete.
 5. The parent then executes the command:
pwd
using an appropriate member of the `exec()` family.
 6. Include appropriate error handling for `fork()` and `exec()`.
-

Expected Output (Sample)

Parent PID: 5201

Child PID : 5202

Parent PID: 5201

Child executing...

Child completed.

(Current working directory displayed)

Deliverables

Each student should submit:

- Source code for all exercises.
- Screenshots showing:
 - Successful compilation.
 - Execution of each program.
 - Process tree using pstree.
 - Output of ps -ef.
- A brief laboratory report (2–3 pages) containing:
 - Objectives.
 - Procedure.
 - Results.
 - Discussion.
 - Challenges encountered.
 - Lessons learned.

Assessment Rubric (20 Marks)

Activity	Marks
Linux process exploration (ps, top, pstree)	2
Process information (getpid(), getppid())	2
fork() programming exercises	4
Parent-child synchronization using wait()	3
exec() implementation	3
Challenge exercises	4
Laboratory report	2
Total	20 Marks

Expected Learning Outcome

At the end of Week 3, students should be able to confidently create and manage Linux processes using fork(), distinguish between parent and child processes, synchronize process execution with wait(), replace process images using the exec() family of system calls, monitor processes using Linux utilities, and apply robust error handling in process-based programs. These practical skills provide the foundation for advanced topics such as inter-process

communication (IPC), process scheduling, multithreading, and concurrent systems programming covered in subsequent weeks.